

Learning conditional loading priors in cEBMF

$p_\theta(L_k \mid L_{<k})$ and a learning algorithm
that reduces to ordinary cEBMF

Methods note for the cEBMF project

16 September 2026

The modeling objective. Learn the conditional distributions $p_\theta(L_k \mid L_{<k})$, and hence the dependent joint prior $p_\theta(L) = \prod_k p_\theta(L_k \mid L_{<k})$, while estimating the latent loadings from the observed matrix. Keep an ordinary learned ash prior on F . The Gaussian-mixture update is implemented with numerical integration. This note distinguishes its exact variational derivation from its computational approximations.

1 The model we want to learn

Let $Y \in \mathbb{R}^{n \times p}$, $L \in \mathbb{R}^{n \times K}$, and $F \in \mathbb{R}^{p \times K}$. Here $L_k = L_{:k}$ means the k th loading column, and $L_{<k} = L_{:,1:(k-1)}$. The observation model is

$$Y_{ij} \mid L, F, \tau \sim \mathcal{N}\left(\sum_{k=1}^K L_{ik} F_{jk}, \tau_{ij}^{-1}\right), \quad (i, j) \in \Omega. \quad (1)$$

The desired autoregressive loading prior is

$$p_\theta(L \mid X) = \prod_{k=1}^K p_{\theta_k}(L_k \mid L_{<k}, X). \quad (2)$$

As in the existing conditional-prior networks, we use the rowwise form

$$p_{\theta_k}(L_k \mid L_{<k}, X) = \prod_{i=1}^n g_{\theta_k}(L_{ik} \mid X_i, L_{i,<k}). \quad (3)$$

The parameters θ_k are shared across rows. If fixed covariates are unavailable, omit X_i . Conditional independence across rows in (3) is a specified architecture; it is not independence across loading columns.

For example, a CGB network learns

$$g_{\theta_k}(dv \mid P) = \pi_{k0}(P)\delta_0(dv) + [1 - \pi_{k0}(P)]\mathcal{N}(dv; \mu_k, s_k^2), \quad P = (X_i, L_{i,<k}).$$

Earlier loadings predict whether a later loading is active. An EMDN can also let earlier loadings predict component means and variances. Multiplying these normalized, acyclic conditionals defines a normalized joint prior. The ordering permits dependencies; it does not uniquely identify a tree.

2 Prior dependence and posterior approximation

The earlier loadings are latent, not observed covariates. A parent loading therefore receives information both from the matrix likelihood and from later loading-prior terms. For two loading coordinates in one row,

$$p(l_1, l_2) = g_1(l_1)g_2(l_2 | l_1) \implies p(l_1 | l_2, Y, F) \propto p(Y | l_1, l_2, F)g_1(l_1)g_2(l_2 | l_1).$$

This posterior feedback follows from the requested joint model itself. It does not introduce a reverse-direction generative prior.

2.1 Mixture labels and the variational family

For clarity, represent a loading mixture by a label Z_{ik} :

$$\begin{aligned} p(Z_{ik} = 0, L_{ik} \in dv | P_{ik}) &= \pi_{k0}(P_{ik})\delta_0(dv), \\ p(Z_{ik} = h, L_{ik} \in dv | P_{ik}) &= \pi_{kh}(P_{ik})\varphi(v; \mu_{kh}(P_{ik}), s_{kh}^2(P_{ik}))dv, \quad h > 0. \end{aligned} \quad (4)$$

Here P_{ik} contains the fixed covariates and the selected earlier parents. These labels only identify existing mixture components. They are not an additional representation or upper latent layer.

Use the variational approximation

$$q(L, Z, F) = \prod_{i,k} q_{ik}(L_{ik}, Z_{ik}) \prod_{j,k} q_{jk}^F(F_{jk}). \quad (5)$$

Each value stays dependent on its own label. The generative prior (2) remains dependent even though this posterior approximation factorizes across loading coordinates. Thus this method *learns the requested conditional prior*, but does not represent posterior covariance between distinct loading coordinates. Section 10 states the alternative if that covariance is also required.

For overlapping Gaussian components, augmenting the labels before making the mean-field approximation can give a different approximation from marginalizing them first. Both describe the same generative marginal model. With fixed parents and no children, the local optimum is the usual normal-means posterior, so augmentation does not spoil the cEBMF limit.

2.2 A single objective

Let $g_{\eta_k}^F$ be the ordinary ash prior on feature column k , with mixture weights estimated during fitting. The unpenalized ELBO is

$$\begin{aligned} \mathcal{L}(q, \theta, \eta, \tau) &= \mathbb{E}_q \log p(Y | L, F, \tau) + H(q) \\ &\quad + \sum_{i,k} \mathbb{E}_q \log g_{\theta_k}(L_{ik}, Z_{ik} | P_{ik}) + \sum_{j,k} \mathbb{E}_q \log g_{\eta_k}^F(F_{jk}). \end{aligned} \quad (6)$$

For the loading terms, densities and entropy are taken with respect to a label-specific reference measure: unit mass at zero for the spike and Lebesgue measure for continuous components. Gaussian densities are not added to spike probability mass at zero.

3 Deriving a loading-coordinate update

3.1 The original cEBMF likelihood calculation

Write $w_{ij} = M_{ij}\tau_{ij}$, where M is the observation mask. Let bars denote first posterior moments, and define

$$\begin{aligned}\bar{R}_{ij,-k} &= Y_{ij} - \sum_{h \neq k} \bar{L}_{ih} \bar{F}_{jh}, \\ a_{ik} &= \sum_j w_{ij} \mathbb{E}_q[F_{jk}^2], & b_{ik} &= \sum_j w_{ij} \bar{R}_{ij,-k} \bar{F}_{jk}.\end{aligned}\tag{7}$$

All sums involving observed Y are over observed entries. Expanding the expected squared error while holding the other variational factors fixed gives the loading-dependent likelihood term

$$\mathbb{E}_{q_{-ik}} \log p(Y \mid L, F, \tau) = -\frac{1}{2}a_{ik}v^2 + b_{ik}v + \text{constant}, \quad v = L_{ik}.\tag{8}$$

The denominator uses $\mathbb{E}[F_{jk}^2]$, not $(\mathbb{E}F_{jk})^2$, as in ordinary cEBMF. F need not be sampled.

3.2 The own-prior and child terms

Define

$$A_{ik}(v, z; \theta_k) = \mathbb{E}_{q(P_{ik})} \log g_{\theta_k}(v, z \mid P_{ik}).\tag{9}$$

Only latent parents are averaged; the external covariates are fixed. Changing v also changes every direct child's prior. Their contribution is

$$D_{ik}(v) = \sum_{c \in \text{ch}(k)} \mathbb{E}_{q(L_{ic}, Z_{ic})q(P_{ic} \setminus L_{ik})} \log g_{\theta_c}(L_{ic}, Z_{ic} \mid P_{ic}[v]).\tag{10}$$

Under the total ordering, every later coordinate is a possible direct child; structurally omitted network inputs create no edge. Holding the other q factors and parameters fixed, the variational coordinate is

$$q_{ik}^*(v, z) \propto \exp\{-a_{ik}v^2/2 + b_{ik}v + A_{ik}(v, z; \theta_k) + D_{ik}(v)\}.\tag{11}$$

This follows by maximizing the terms of (6) involving q_{ik} : expected log joint plus its entropy.

Two necessary corrections to plug-in conditioning. Average the *log prior* over uncertain parents and include the children's prior terms. In general, $\mathbb{E} \log g(v \mid P) \neq \log g(v \mid \mathbb{E}P)$ and $\mathbb{E} \log g(v \mid P) \neq \log \mathbb{E}g(v \mid P)$. Using fitted earlier loadings as fixed inputs does not perform these steps.

4 Learning the conditional prior and posterior together

Write the exponent in (11) as $\Psi_{ik}(v, z; \theta_k)$, and define its normalizer

$$\mathcal{Z}_{ik}(\theta_k) = \sum_z \int \exp\{\Psi_{ik}(v, z; \theta_k)\} d\nu_z(v). \quad (12)$$

The network parameters are distinct for different loading columns. Consequently D is fixed when optimizing θ_k , even though it varies with the candidate loading v . The Gibbs variational identity gives

$$\max_{q_{ik}} \{\mathbb{E}_{q_{ik}} \Psi_{ik} + H(q_{ik})\} = \log \mathcal{Z}_{ik}(\theta_k). \quad (13)$$

The joint prior-and-posterior update is therefore

$$\boxed{\theta_k^{\text{new}} \in \arg \max_{\theta_k} \sum_i \log \mathcal{Z}_{ik}(\theta_k), \quad q_{ik}^{\text{new}} = q_{ik}^*(\cdot; \theta_k^{\text{new}}).} \quad (14)$$

The neural prior is learned from all rows using the same ELBO as the loading inference. An exact block optimizer cannot decrease that ELBO. Approximate integration and finite neural optimization require their own accuracy controls; they do not inherit an automatic monotonicity guarantee.

4.1 Proof of the reduction to ordinary cEBMF

With no latent-covariate edges, there are neither uncertain parents nor child messages:

$$A_{ik}(v, z; \theta_k) = \log g_{\theta_k}(v, z \mid X_i), \quad D_{ik}(v) = 0.$$

For $a = a_{ik} > 0$, set $x = b_{ik}/a$ and $s^2 = 1/a$. Then

$$e^{-av^2/2+bv} = \sqrt{2\pi/a} e^{b^2/(2a)} \varphi(x; v, 1/a).$$

After summing over the component label,

$$\begin{aligned} \log \mathcal{Z}_{ik}(\theta_k) &= \log \left\{ \int \varphi(x; v, s^2) g_{\theta_k}(dv \mid X_i) \right\} \\ &\quad + \frac{1}{2} \log(2\pi/a) + b^2/(2a). \end{aligned} \quad (15)$$

The last two terms do not depend on θ_k . Thus (14) is **exactly the original cEBNM marginal-likelihood fit**, and (11) gives its posterior. Feature and noise updates below also reduce to their ordinary cEBMF forms.

In code, use this analytic limit and call the existing cEBNM solver with the same options, initialization and update schedule. A single complete-data EM M-step, even using infinitely many exact posterior draws, is not generally the same update as this marginal-likelihood fit.

For $a = b = 0$ use (11) directly. Also, do not renormalize $\exp(A)$ as an ordinary prior and discard its normalizer: with uncertain parents that normalizer generally depends on θ_k .

5 Gaussian mixtures and exact zero spikes

Suppress the row and loading indices. For each continuous component h , the expected log own prior is quadratic in v . Define

$$\begin{aligned}\alpha_h &= a + \mathbb{E}_P[s_h(P)^{-2}], \\ \beta_h &= b + \mathbb{E}_P[\mu_h(P)s_h(P)^{-2}], \\ \gamma_h &= \mathbb{E}_P \log \pi_h(P) - \frac{1}{2} \mathbb{E}_P \log(2\pi s_h(P)^2) - \frac{1}{2} \mathbb{E}_P[\mu_h(P)^2 s_h(P)^{-2}].\end{aligned}\quad (16)$$

Assume these expectations are finite and $\alpha_h > 0$. Put $m_h = \beta_h/\alpha_h$, $V_h = 1/\alpha_h$, and

$$I_{rh} = \int v^r \varphi(v; m_h, V_h) e^{D(v)} dv, \quad r = 0, 1, 2. \quad (17)$$

Completing the square gives unnormalized component masses

$$\begin{aligned}W_h &= \exp\{\gamma_h + \beta_h^2/(2\alpha_h)\} \sqrt{2\pi/\alpha_h} I_{0h}, \quad h > 0, \\ W_0 &= \exp\{\mathbb{E}_P \log \pi_0(P) + D(0)\}.\end{aligned}\quad (18)$$

The zero-spike likelihood factor is one in the natural-parameter form $e^{-av^2/2+bv}$. There is no omitted Gaussian evidence constant in W_0 . Normalize with $r_h = W_h/\sum_t W_t$. Then

$$\begin{aligned}q(v \mid z = h) &= \frac{\varphi(v; m_h, V_h) e^{D(v)}}{I_{0h}}, \quad h > 0, \\ \mathbb{E}_q v &= \sum_{h>0} r_h I_{1h}/I_{0h}, & \mathbb{E}_q v^2 &= \sum_{h>0} r_h I_{2h}/I_{0h}.\end{aligned}\quad (19)$$

Omit the spike for an unspiked EMDN. Compute component masses in log space. With deterministic parents and $D = 0$ these are the ordinary analytic Gaussian-mixture normal-means formulas.

5.1 How to calculate the child message

For child c , let $r_{ch} = q(Z_c = h)$, $u_{ch} = \mathbb{E}(L_c \mid Z_c = h)$, and $T_{ch} = \mathbb{E}(L_c^2 \mid Z_c = h)$. A Gaussian component contributes

$$J_{ch}(P) = -\frac{1}{2} \log(2\pi s_{ch}^2(P)) - \frac{T_{ch} - 2\mu_{ch}(P)u_{ch} + \mu_{ch}(P)^2}{2s_{ch}^2(P)}. \quad (20)$$

Consequently,

$$D(v) = \sum_{c \in \text{ch}(k)} \sum_h r_{ch} \mathbb{E}_{P_c \setminus v} [\log \pi_{ch}(P_c[v]) + J_{ch}(P_c[v])], \quad (21)$$

with $J_{c0} = 0$ for the fixed zero atom. For CGB with a global slab, J is constant in v and drops from the v -dependent tilt. For EMDN, its means and variances depend on parents, so J must remain. Child labels and values can be integrated using their probabilities and moments rather than sampled.

6 A two-loading example of the desired dependency

As a transparent illustration, suppose the loadings are strictly binary:

$$L_1 \sim \text{Bernoulli}(\rho), \quad L_2 \mid L_1 \sim \text{Bernoulli}\{s(\alpha + \beta L_1)\}, \quad s(t) = \text{sigmoid}(t). \quad (22)$$

This directly models $p(L_2 \mid L_1)$. Under $q(L_1)q(L_2)$, let $r_1 = q(L_1 = 1)$ and $r_2 = q(L_2 = 1)$. Since $v^2 = v$ for binary v , the likelihood log-odds contribution is $b_k - a_k/2$. The child update is

$$\text{logit}(r_2^{\text{new}}) = b_2 - a_2/2 + \alpha + \beta r_1. \quad (23)$$

For the parent, write $s_0 = s(\alpha)$ and $s_1 = s(\alpha + \beta)$. Its update contains feedback from the inferred child activation:

$$\begin{aligned} \text{logit}(r_1^{\text{new}}) &= b_1 - a_1/2 + \text{logit}(\rho) \\ &\quad + r_2 \log(s_1/s_0) + (1 - r_2) \log\{(1 - s_1)/(1 - s_0)\}. \end{aligned} \quad (24)$$

If $\beta > 0$, a confidently active child tends to favor parent activation. If $\beta = 0$, the child correction vanishes and both equations reduce to ordinary independent Bernoulli-prior normal-means updates.

This example distinguishes three objects: the learned conditional prior in (22), the posterior approximation r_1, r_2 , and the posterior feedback required by that prior. A factorized q does not remove the conditional relationship being learned in p.

7 Keeping the feature update as ordinary ash

Under the scalar-factorized approximation (5), set

$$\begin{aligned} a_{jk}^F &= \sum_i w_{ij} \mathbb{E}_q[L_{ik}^2], \\ b_{jk}^F &= \sum_i w_{ij} \left(Y_{ij} - \sum_{h \neq k} \bar{L}_{ih} \bar{F}_{jh} \right) \bar{L}_{ik}. \end{aligned} \quad (25)$$

Fit the existing ash normal-means model at b_{jk}^F/a_{jk}^F , with SE $(a_{jk}^F)^{-1/2}$. Update its mixture weights and posterior moments at each fitting iteration. Its configured penalty and other options are preserved.

This is the original feature-side empirical Bayes update. Merely drawing F from a prior fitted during initialization would not implement this learning algorithm. No HMM or feature-side neural model is needed for this derivation.

8 Noise, entropy and regularization

8.1 Observation noise

Under (5), the expected squared residual is

$$\begin{aligned}\mathbb{E}_q[(Y_{ij} - L_i^T F_j)^2] &= (Y_{ij} - \bar{L}_i^T \bar{F}_j)^2 \\ &\quad + \sum_k \{\mathbb{E}[L_{ik}^2] \mathbb{E}[F_{jk}^2] - \bar{L}_{ik} \bar{F}_{jk}\}.\end{aligned}\quad (26)$$

For unknown constant precision,

$$\tau^{\text{new}} = \frac{|\Omega|}{\sum_{ij \in \Omega} \mathbb{E}_q[(Y_{ij} - L_i^T F_j)^2]}.\quad (27)$$

The corresponding row- or column-specific versions use the same sums restricted to their groups. Known noise remains fixed.

8.2 Entropy of the actual variational approximation

For a continuous loading component in (19),

$$H_h = \frac{1}{2} \log(2\pi V_h) + \frac{\mathbb{E}_h[(v - m_h)^2]}{2V_h} - \mathbb{E}_h[D(v)] + \log I_{0h}.\quad (28)$$

The value-and-label entropy is $-\sum_h r_h \log r_h + \sum_{h>0} r_h H_h$. The atom has zero within-component entropy. For F, use the ordinary EBNM identity to compute its prior KL contribution. Thus the ELBO can be evaluated for the actual q used by the algorithm, rather than a separate distribution fitted afterward to sampler marginals.

When later coordinates or priors change, a previously updated q must stay fixed until its next explicit update. A stored tilted density cannot silently reevaluate itself using newly changed parent distributions or neural parameters. Its representation must preserve the state that defined it.

8.3 A consistent sparsity penalty

First validate the unpenalized case. The existing neural spike regularizer can then be included as

$$R(q, \theta) = \sum_{i,k} (\lambda_k - 1) \mathbb{E}_q \log \pi_{k0}(P_{ik}), \quad \lambda_k \geq 1.\quad (29)$$

The own penalty is constant in the candidate L_{ik} , so it can be added outside the log normalizer during θ_k optimization. A child's penalty depends on L_{ik} and must be included in $D(v)$. Without latent parents these become the fixed-covariate parameter penalties, preserving the ordinary penalized cEBNM limit.

This is a regularized ELBO, or a bound for an augmented observation model; it is not the unpenalized model's evidence. The ash global mixture-weight penalty has its existing, separate semantics and should be preserved.

9 The learning algorithm

One cEBMF iteration. Retain the original residual-based L_k/F_k schedule. Extend only the loading fit to account for uncertain parents and child feedback.

1. Initialize L, F and their moments using the existing cEBMF scheme. Preserve supplied factors and their scaling. There is no universal statistical justification for thresholding initial loadings at 0.5.
2. For $k = 1, \dots, K$:
 - (a) Calculate a^L, b^L from current variational moments using (7).
 - (b) If k has no latent parents or children, call its original cEBNM solver. Otherwise compute (9)–(10) and jointly fit the conditional prior and posterior through (14).
 - (c) Store loading component probabilities, moments and the fixed variational representation needed for later expectations and entropy.
 - (d) Update F_k with the original ash fit using (25), and refresh residuals.
3. Update unknown noise using the expected residuals and evaluate the same variational objective.
4. Repeat until the chosen iteration limit or convergence criterion.

The target remains $\prod_k p_\theta(L_k \mid L_{<k})$. This variational algorithm does not require a final full-matrix MCMC burn-in and sampling stage.

9.1 Where numerical approximation remains

Each integral over the target loading in (17) is one-dimensional. Adaptive quadrature or Gauss–Hermite integration around the Gaussian base components can evaluate it. Expectations over several uncertain parents can require separate quadrature or Monte Carlo integration. Keep parent integration nodes or draws fixed during a local parameter fit, and check results by increasing integration accuracy.

Sharp child constraints can make these calculations demanding. A practical implementation must assess runtime and approximation error; the derivation does not guarantee a fast neural optimizer or a predictive improvement. When there are no latent edges, however, use the analytic cEBNM route and avoid introducing numerical approximation unnecessarily.

9.2 Compatibility and convergence requirements

The no-edge route should preserve the old prior options, initialization, RNG behavior, L_k/F_k schedule, noise updates and pruning policy. With a hierarchy, deleting a factor changes child input identities and needs an explicit graph/network policy. Fixing rank is reasonable for an initial controlled comparison only when the comparator uses the same rank.

The exact variational argument gives nondecrease of the ELBO for exact block fits. Existing approximate scalar solvers and finite neural updates are not all exact maximizers; behavioral compatibility must not be confused with an unconditional monotonicity theorem.

10 If within-row posterior dependence is also required

The same conditional-prior model can also be fitted with posterior covariance between loading coordinates. For this additional inference choice, consider

$$q(L, F) = \prod_i q_i(L_i) \prod_{j,k} q_{jk}^F(F_{jk}).$$

The optimal row coordinate is

$$q_i^*(L_i) \propto p_\theta(L_i | X_i) \exp\{-\frac{1}{2}L_i^T H_i L_i + h_i^T L_i\}, \quad (30)$$

where

$$H_i = \sum_j w_{ij} \mathbb{E}[F_j F_j^T], \quad h_i = \sum_j w_{ij} Y_{ij} \mathbb{E}[F_j].$$

This retains the same conditional-prior model, but the feature numerator must now use cross moments:

$$b_{jk}^F = \sum_i w_{ij} \left[Y_{ij} \mathbb{E}[L_{ik}] - \sum_{h \neq k} \mathbb{E}[L_{ik} L_{ih}] \mathbb{E}[F_{jh}] \right]. \quad (31)$$

Relative to (25), the extra term is

$$- \sum_i w_{ij} \sum_{h \neq k} \text{Cov}(L_{ik}, L_{ih}) \bar{F}_{jh}.$$

Thus one cannot sample a correlated L block and pass only marginal means and variances into an unchanged mean-residual feature update.

Full row blocks do not automatically recover mean-field cEBMF when prior edges vanish: the likelihood can still correlate loading coordinates. For exact reduction, define blocks by connected components of the prior graph and split them into singleton columns in the no-edge case. Singleton blocks recover ordinary cEBNM; larger blocks require a multivariate profile objective.

11 Verification and status

The repository script `output/review_joint_1/verify_replacement_derivation.py` checks:

- Gaussian-mixture probabilities, moments and log normalizers in the no-parent/no-child limit: maximum absolute error 2.0×10^{-15} .
- Independent and dependent binary coordinates against exhaustive enumeration: maximum probability error 1.2×10^{-16} , with nonnegative exact coordinate ELBO gains.
- The profile objective against the directly optimized enumerated ELBO at four parameter settings: their difference is constant to 6.7×10^{-16} .
- The structured feature numerator: a correlated binary example gives the correct value 0; multiplying marginal means incorrectly gives 0.25.

The implementation in `cebmf/_conditional.py` additionally has analytic profile-gradient, no-edge compatibility and missing-modality regression tests. Its integration approximations and finite training still require sensitivity checks and comparative empirical evaluation.

12 Adversarial numerical review and device handling

Review added 16 September 2026. The target remains $p(L_k \mid L_{<k})$. Removing latent-covariate edges gives the same normal-means empirical Bayes subproblem; the implementation dispatches this case to the ordinary cEBMF solver. This algebraic statement does not assert that every finite neural optimization attains the subproblem optimum.

12.1 A correct formula can be an unstable implementation

Completing a Gaussian square using its natural parameters subtracts terms of order μ^2/s^2 . In float32 this failed for a prior with spike weight 0.2 and slab $\mathcal{N}(10, 10^{-8})$, with likelihood statistics $a = 2, b = 3$. The old numerical profile returned a posterior mean near 0.0125; the independent analytic posterior is approximately 1.59×10^{-29} . This is cancellation error, not Monte Carlo variability.

For each continuous component, compute instead

$$t = \mathbb{E}[s(P)^{-2}], \quad c = \frac{\mathbb{E}[\mu(P)s(P)^{-2}]}{t}, \quad (32)$$

$$h = \mathbb{E} \log \pi(P) - \frac{1}{2} \mathbb{E} \left[\log(2\pi s(P)^2) + \frac{(\mu(P) - c)^2}{s(P)^2} \right]. \quad (33)$$

Then the expected log prior is exactly

$$A(v) = h - \frac{1}{2}t(v - c)^2.$$

The spread in this expression is evaluated as an average of squared centered differences, never as a difference of large second moments. The height h retains the parameter-dependent normalization of $\exp(\mathbb{E} \log g)$; dropping it would change empirical Bayes learning.

Write $\alpha = a + t$, $r = t/\alpha$. The likelihood-completed Gaussian has mean $m = rc + b/\alpha$, variance $V = 1/\alpha$, and log mass

$$B = h + r(-ac^2/2 + bc) + \frac{b^2}{2\alpha} + \frac{1}{2} \log \frac{2\pi}{\alpha}. \quad (34)$$

Its tilted contribution is $\exp(B)\mathbb{E}_{N(m,V)} \exp D(v)$. For the zero atom the corresponding log mass before the child tilt is $\mathbb{E} \log \pi_0(P)$. These are algebraic rearrangements of the earlier derivation, not a different statistical model.

At a standard-normal quadrature node x_j , evaluate the slab log density as $-\frac{1}{2}\{\log(2\pi V) + x_j^2\}$. Combining it with $B + D(v) - \log Z$ gives the continuous-and-label log posterior used for entropy. This avoids another unstable natural-parameter polynomial. For child feedback use

$$\mathbb{E}[(L_c - \mu(P))^2 \mid Z_c = h] = \text{Var}(L_c \mid Z_c = h) + (\mathbb{E}[L_c \mid Z_c = h] - \mu(P))^2,$$

computing the within-component variance directly from centered nodes.

12.2 What was checked, and what was not established

Independent checks include exhaustive binary enumeration, analytic mixture posteriors (including zero likelihood precision), and a dense trapezoidal integral for a nonlinear logistic child tilt. Evidence, moments, entropy, and parameter gradients agree at the stated test tolerances. Direct child integration also checks the sparsity penalty and missing-branch mask. The ordinary

spike-normal posterior now uses log-domain component odds; normal-mixture likelihoods are no longer silently clipped in the tails.

These tests do not prove convergence of finite quadrature for every neural prior. Multiple-parent expectations use a finite fixed Sobol approximation; the stored quadrature representation also approximates each continuous coordinate. Local profile checkpoint selection is not a proof of global ELBO ascent under those approximations. The posterior is still factorized across loading coordinates. Better prior flexibility can lose predictive accuracy through optimization, approximation, or overfitting.

The saved tree benchmark predates these numerical fixes. Inspecting its checkpoints confirms float32 observations and loadings; the earlier claim of float64 fitting was incorrect. The narrow-slab bug could affect those fits, but attributing the performance gap to it requires matched reruns. Original source hashes and measurements are retained and have not been relabeled as results of the revised implementation.

12.3 CUDA contract and its validation limit

Loading moments, networks, quadrature states, optimizer state, residuals, and objective histories are kept on the selected device. Conditional checkpoint selection and mixture-EM convergence use device tensors. Observation indices and prediction tables are cached. Sobol tables are constructed once on CPU and transferred at graph setup. Ordinary CUDA neural solvers batch with device indices.

ASH’s automatically sized scale grid needs one host scalar for its length; its amplitudes stay on device. Supplying a fixed `scales` tensor to the ash prior removes that dynamic-size decision and still refits weights on every sweep. Fixed-grid choice is a modeling choice, not an automatically equivalent replacement for an adaptive grid. Rank pruning, boundary input validation, plotting, and explicit reporting may also synchronize.

The development environment has PyTorch 2.11.0+cpu and no CUDA runtime. CUDA-only tests cover complete warmed sweeps, partially paired modalities, optimizer-state residency, CPU/GPU numerical agreement, profiler transfers, and retained memory. They are skipped here, not reported as GPU passes. Hardware validation remains required before making a measured GPU speed or memory claim. The deprecated experimental sampler and optional L-BFGS route are outside the synchronization-free conditional/ash-EM contract.

Reference. Denault et al. (2025), *Covariate-moderated Empirical Bayes Matrix Factorization*, Section 3.3 and its variational derivation. [arXiv:2505.11639](https://arxiv.org/abs/2505.11639). The conditional-prior extension and reduction above are derived for this project.